

Prague University of Economics and Business
Faculty of Informatics and Statistics



A Data Integration Framework for Enterprise Application Security

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence

Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus

Supervisor: Ing. Aleš Kotuč

Prague, month YYYY

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

Introduction	11
Motivation	11
Objective	11
Methodology	11
Structure	11
1 Analysis and Requirements	12
1.1 Application Inventory	12
1.1.1 Application Portfolio Management	12
1.1.2 Configuration Management Databases	13
1.1.3 Integration Options	13
1.2 Software Development	15
1.2.1 Source Code Management	16
1.2.2 Continuous Integration and Delivery	16
1.2.3 Issue Tracking	17
1.3 Application Security	19
1.3.1 Static Application Security Testing	19
1.3.2 Software Composition Analysis	19
1.3.3 Secret Detection	19
1.3.4 Dynamic Testing and Penetration Testing	19
1.3.5 Container Image Scanning	19
1.3.6 Infrastructure as Code Security	19
1.3.7 Runtime Security Monitoring	19
1.4 Data Engineering	19
1.4.1 Data Platform Architecture	19
1.4.2 Data Integration Patterns	19
1.4.3 Domain Data Model	19
1.5 Related Work and Gap Analysis	19
1.5.1 Existing Approaches	19
1.5.2 Databricks Lakewatch	19
1.5.3 Comparative Analysis and Research Gap	19
2 Framework	20
2.1 Solution Architecture	21
2.1.1 Component Design	21
2.1.2 Medallion Architecture	21
2.1.3 Technology Stack	21
2.2 Data Model	21
2.2.1 Schema Patterns	21

2.2.2	Entity Model	21
2.2.3	Aggregation Model	21
2.3	Connector Framework	21
2.3.1	Connector Abstraction	21
2.3.2	Ingestion Patterns	21
2.3.3	Transformation Patterns	21
2.4	Analytics and Serving Layer	21
2.4.1	Aggregation Patterns	21
2.4.2	ML Workflows	21
2.4.3	Serving Patterns	21
2.5	Deployment and Engineering Practices	21
2.5.1	Deployment Strategy	21
2.5.2	Project Structure	21
2.5.3	Testing Strategy	21
3	Reference Implementation	22
3.1	Environment and Deployment	23
3.1.1	Workspace Setup	23
3.1.2	Silver Schema	23
3.1.3	Project Structure and CI/CD	23
3.2	Connectors	23
3.2.1	ServiceNow	23
3.2.2	GitHub	23
3.2.3	GitLab	23
3.2.4	SonarQube	23
3.2.5	Semgrep	23
3.2.6	Dependency-Track	23
3.2.7	TruffleHog	23
3.2.8	Vulnerability Enrichment	23
3.3	Analytics and Serving	23
3.3.1	Application-Repository Mapping	23
3.3.2	Application Risk Scoring	23
3.3.3	Remediation and Compliance Metrics	23
3.3.4	Vulnerability Trends	23
3.3.5	Risk Prediction Model	23
3.3.6	Serving Layer	23
3.4	Testing and Validation	23
	Conclusion	24
	Thesis Outcomes and Contributions	24
	Limitations	24
	Future Work	24
	References	25

A	Generative AI Use Disclosure	28
A.1	Text Content	28
A.2	Diagrams and Figures	28
A.3	Source Code	28
A.4	Requirements Specification	28

List of Figures

List of Tables

List of source codes

Introduction

Motivation

Objective

Methodology

Structure

1. Analysis and Requirements

This chapter surveys the literature and analyzes the functional domains relevant to building an application security data platform. Each section covers one domain, combining theoretical foundations with practical tool and Application Programming Interface (API) analysis. The focus is on reviewing existing knowledge, observing source system interfaces, data models, and integration patterns, not on prescribing a solution. Technology choices for the target platform are deferred to [Chapter 2](#) and [Chapter 3](#).

[section 1.1](#) establishes the business context underlying all security findings. [section 1.2](#) and [section 1.3](#) examine the software development and security tooling landscapes as data sources. [section 1.4](#) covers the data engineering patterns that underpin the framework's design. Finally, [section 1.5](#) surveys existing solutions and identifies the gap this thesis addresses.

1.1 Application Inventory

The application inventory is the foundational data source for any application security data platform. It establishes the business context for all security findings. Organizations typically manage this inventory in a Configuration Management Database (CMDB) or a custom-built catalog. Without a reliable catalog of business applications and their mapping to technical assets, a vulnerability discovered in a repository or a running service cannot be attributed to an owner, assessed for business impact, or prioritized against organizational risk criteria.

1.1.1 Application Portfolio Management

Enterprise organizations maintain catalogs of their business applications with metadata such as application name, description, ownership information, lifecycle status, and relationships to other assets (AXELOS, [2019](#)).

These registries also capture security-relevant attributes. Information security is commonly assessed along three dimensions known as the Confidentiality, Integrity, and Availability (CIA) triad: confidentiality (preventing unauthorized disclosure), integrity (preventing unauthorized modification), and availability (ensuring authorized access) (Stallings & Brown, [2024](#)). Business impact assessments evaluate all three CIA dimensions to produce criticality ratings, commonly structured as tiers: Tier 1 for mission-critical applications, Tier 2 for important supporting systems, and Tier 3 for non-critical internal tools. Separately, regulatory scope flags identify which compliance frameworks (e.g., PCI DSS, GDPR, SOX) apply to a given application.

The most important attribute from a data integration perspective is the mapping between business applications and their technical assets. A single business application may span multiple source code repositories, microservices, infrastructure components, and deployment environments. This mapping serves as the glue connecting technical security findings to business context. When the framework ingests a vulnerability from a Static Application Security Testing (SAST) scanner linked to a specific repository, the application inventory mapping determines which business application is affected, who owns it, and how critical it is. This enables the risk-aware prioritization that stakeholders require.

1.1.2 Configuration Management Databases

The most common CMDB platform in enterprise environments is ServiceNow, holding over 44% of the global IT Service Management (ITSM) market share (Gartner, 2024). It serves as the reference platform for this thesis. A CMDB organizes all managed assets as Configuration Items (CIs), each with attributes describing its type, status, and ownership (AXELOS, 2019).

ServiceNow arranges CIs in a class hierarchy rooted at the `cmdb_ci` base table. Each subclass inherits base attributes (name, `sys_id`, operational status, environment) and adds domain-specific fields. For application inventory purposes, the most relevant class is `cmdb_ci_app1` (Application), which extends the base CI with attributes such as version, business unit, used-for classification, and references to the owning support group (ServiceNow, 2024a).

Beyond the application record itself, the CMDB captures relationships between CIs through a dedicated relationship table (`cmdb_rel_ci`). These relationships model dependencies such as “runs on” (application to server), “depends on” (application to database), and “hosted on” (application to cloud service). For the framework, the most valuable relationships are those linking business applications to technical assets that correspond to entities in the security data: repository names, deployment targets, and infrastructure components. ServiceNow also allows organizations to define custom attributes and relationship types, so the specific fields available for ingestion may vary between deployments.

1.1.3 Integration Options

Integrating CMDB data into an external data platform requires choosing between two strategies: pulling data from ServiceNow via its APIs, or pushing data from a custom ServiceNow application to an external target.

ServiceNow APIs

ServiceNow exposes two complementary Representational State Transfer (REST) APIs for CMDB data extraction. The Table API provides generic CRUD operations against any ServiceNow table through the endpoint `/api/now/table/{tableName}` (ServiceNow, 2024c). Responses are JavaScript Object Notation (JSON)-formatted, with support for server-side filtering (`sysparm_query`), field selection (`sysparm_fields`), and pagination via `sysparm_limit` and `sysparm_offset` (default maximum of 10 000 records per request). The `sysparm_display_value` parameter resolves reference fields to human-readable names instead of internal `sys_id` values.

The CMDB Instance API (`/now/cmdb/instance/{className}`) provides a CMDB-aware alternative that understands the CI class hierarchy (ServiceNow, 2024a). It can retrieve a CI with its relationships in a single call, reducing the number of API calls needed to reconstruct the application-to-asset mapping. However, it is more constrained in its query capabilities than the Table API.

Both APIs require authentication, typically through Basic Authentication with a service account granted the `rest_service` or `cmdb_read` role, or through OAuth 2.0 for more security-conscious deployments. Rate limits vary by instance configuration and are governed by platform transaction quotas rather than per-endpoint limits. For incremental data extraction, the `sys_updated_on` timestamp field present on all records serves as a reliable high-water mark, allowing consumers to query only records modified since the last extraction.

Any pull-based integration must handle authentication, pagination, rate limiting, and incremental state management. Several integration libraries and platform-native connectors abstract these concerns; the specific tooling choice depends on the target data platform and is discussed in [subsection 3.2.1](#).

ServiceNow Application

An alternative to pulling data is to push it from the source. ServiceNow supports building scoped applications that run on the platform itself. A custom application can use Outbound REST Messages to send CMDB data to an external target (e.g., a cloud storage bucket or a REST endpoint) on a schedule defined through Scheduled Jobs or Flow Designer. The application can also expose a Scripted REST API (ServiceNow, 2024b), a custom endpoint that shapes the data exactly as the consumer needs it, pre-joining application records with their relationships and custom attributes in a single response.

The advantage is full access to ServiceNow's internal scripting capabilities, relationship traversal, and business logic without external API call overhead. The disadvantage is that development and maintenance happen inside the ServiceNow platform, requiring

ServiceNow development expertise and a separate deployment lifecycle. Changes to the CMDB schema or extraction logic must be coordinated across two platforms. This approach is best suited for organizations with mature ServiceNow development teams that prefer to own the data export logic on the source side.

Integration Challenges

In implementation phase, the application inventory presents the most significant integration challenge among all data sources because it is the least standardized. Security testing tools converge around common API patterns and output formats, but each organization structures its application inventory differently: the hierarchy of applications, the granularity of asset mapping, and the attributes captured vary substantially (AXELOS, 2019). This makes the application inventory connector the hardest to generalize and the one most likely to require implementation-specific customization.

Data quality in CMDBs is a persistent concern (Williams & Gonzalez, 2021). Application records may be incomplete (missing ownership or criticality assignments), outdated (reflecting decommissioned applications still listed as active), or inconsistently maintained across business units. Any integration must account for these quality issues through validation rules and default values that surface problems without blocking the data pipeline.

Despite these difficulties, the application inventory is indispensable. Without the business context it provides, security findings lack the organizational meaning needed for effective prioritization and governance.

1.2 Software Development

Modern software is built collaboratively from source code, organized in repositories, and deployed through automated pipelines (Pressman & Maxim, 2019; Sommerville, 2015). While section 1.1 established the business context for security findings, the platforms that support software development provide the technical entities around which those findings are organized. Repositories, build pipelines, and issue trackers form the second major data source category for the framework.

These platforms share common integration patterns that simplify connector design. All expose REST APIs with JSON responses as the primary integration surface. Authentication relies on token-based mechanisms: OAuth tokens, Personal Access Tokens (PATs), or service account credentials. Pagination follows either offset-based or cursor-based patterns, and all platforms impose rate limits that consumers must handle through backoff strategies. This consistency enables shared connector logic for authentication, pagination, rate limiting, and error recovery across tools.

1.2.1 Source Code Management

Chacon and Straub (2014) provide a thorough treatment of Git, covering branching strategies, merging, and distributed workflows, while Skoulikari (2024) offers a more recent visual introduction. Winters et al. (2020) extend this perspective to organizational scale, describing how large engineering teams manage repositories with thousands of contributors. For this thesis, the repository is the primary entity around which security findings are grouped, as detailed in the data model in Chapter 2.

Development teams host their Git repositories on cloud-based Source Code Management (SCM) platforms. GitHub is the dominant platform, with over 150 million developers and adoption by 92% of Fortune 100 companies (GitHub, 2025). The 2025 Stack Overflow Developer Survey reports GitHub at 81% adoption for code collaboration, followed by GitLab at 36% (Stack Overflow, 2025). Bitbucket and Azure DevOps serve smaller but significant enterprise segments. Most large organizations use more than one platform, whether through acquisitions, team preferences, or regulatory separation, making multi-platform ingestion a practical requirement.

Beyond source code, SCM platforms expose metadata relevant to security governance. Repository attributes include the primary programming language, creation and last activity dates, visibility settings (public or private), and archive status. Branch protection rules indicate the maturity of the development process. Team and contributor assignments establish ownership relationships that feed into the application-to-team mapping described in section 1.1.

These platforms share common API patterns. GitHub exposes two complementary APIs (GitHub, 2024): a REST API (v3) with resource-oriented endpoints and page-based pagination, and a GraphQL API (v4) that enables selective field retrieval with cursor-based pagination. Both use the same authentication mechanisms (OAuth tokens or PATs) and share a rate limit of 5 000 requests per hour. GitLab offers a similar dual REST/GraphQL surface (GitLab, 2024). Azure DevOps provides a REST API authenticated through Azure Active Directory, with comparable response formats and pagination. Across all platforms, authentication is token-based and responses are JSON-formatted, enabling shared connector logic for authentication, pagination, and rate limit handling.

1.2.2 Continuous Integration and Delivery

Software Development Lifecycle (SDLC) practices have shifted from sequential models to iterative methodologies and DevOps. Forsgren et al. (2018) present empirical evidence that high-performing teams deploy more frequently with shorter lead times, achieved through Continuous Integration and Continuous Delivery (CI/CD) automation. Kim et al. (2021) complement this with practical guidance for implementing DevOps pipelines at scale. The CI/CD pipeline, an automated sequence of build, test, and deploy stages, is relevant to this thesis

because each stage can integrate security tools whose findings the framework consolidates.

GitHub Actions is the most widely adopted CI/CD platform, used by 62% of developers for personal projects and 41% in organizational settings (JetBrains, 2025). Jenkins remains prevalent in enterprises at 28% organizational adoption, followed by GitLab CI at 19%. Notably, 32% of organizations use two or more CI/CD tools, and 9% use at least three, reinforcing the multi-platform integration challenge observed for SCM platforms.

CI/CD platforms provide contextual data about when and how security scans run. Pipeline definitions reveal which security tools are integrated into the build process. Build results indicate whether security gates passed or failed. Execution metadata records which commit was scanned, when the scan ran, and how long it took. This information serves as operational context rather than a primary finding source: knowing that a repository's last security scan ran three months ago, or that a scan consistently fails, signals coverage gaps and tool health that complement the findings from security tools.

Integration patterns vary more across CI/CD platforms than across SCM tools. GitHub Actions exposes build data through its REST API with JSON responses, consistent with the broader GitHub API suite. Jenkins uses an Extensible Markup Language (XML)-based API with a different authentication model. Azure Pipelines follows the Azure DevOps REST conventions. Despite these differences, the retrieved data is structurally similar: pipeline identifiers, run statuses, timestamps, and associated commit references.

1.2.3 Issue Tracking

Issue tracking systems are relevant to the framework because they track remediation status. When a security finding is assigned for remediation, an issue is typically created in the team's tracker, either manually or through automation. The framework must read issue status to determine whether a finding is under active remediation, who is responsible, and whether it has been resolved within the required Service Level Agreement (SLA) window.

Jira is the dominant issue tracking platform in enterprise environments. The 2025 Stack Overflow Developer Survey reports Jira at 46% adoption, behind only GitHub (81%) and ahead of GitLab (36%) for code documentation and collaboration tools (Stack Overflow, 2025). Azure DevOps Work Items serves organizations within the Microsoft ecosystem. GitHub Issues and GitLab Issues provide lightweight built-in tracking tightly coupled with their respective SCM platforms. As with SCM and CI/CD tools, many organizations use multiple trackers across different teams.

These platforms expose REST APIs with JSON responses and paginated results. Jira provides Jira Query Language (JQL), a query language for precise filtering by project, status, labels, or custom fields, enabling retrieval of only security-relevant issues without downloading entire project backlogs. GitHub and GitLab expose issues through the same API surfaces used for repository data. Most trackers also support webhooks for real-time event notifications

when issue status changes, reducing reliance on periodic polling.

The integration with issue trackers is bidirectional. The framework reads remediation status and may also create new issues when findings require attention. This bidirectional flow introduces complexity around idempotency (avoiding duplicate issue creation) and state synchronization (keeping the framework's view consistent with the tracker's current state).

1.3 Application Security

1.3.1 Static Application Security Testing

1.3.2 Software Composition Analysis

1.3.3 Secret Detection

1.3.4 Dynamic Testing and Penetration Testing

1.3.5 Container Image Scanning

1.3.6 Infrastructure as Code Security

1.3.7 Runtime Security Monitoring

1.4 Data Engineering

1.4.1 Data Platform Architecture

1.4.2 Data Integration Patterns

1.4.3 Domain Data Model

1.5 Related Work and Gap Analysis

1.5.1 Existing Approaches

1.5.2 Databricks Lakewatch

1.5.3 Comparative Analysis and Research Gap

2. Framework

2.1 Solution Architecture

2.1.1 Component Design

2.1.2 Medallion Architecture

2.1.3 Technology Stack

2.2 Data Model

2.2.1 Schema Patterns

2.2.2 Entity Model

2.2.3 Aggregation Model

2.3 Connector Framework

2.3.1 Connector Abstraction

2.3.2 Ingestion Patterns

2.3.3 Transformation Patterns

2.4 Analytics and Serving Layer

2.4.1 Aggregation Patterns

2.4.2 ML Workflows

2.4.3 Serving Patterns

2.5 Deployment and Engineering Practices

2.5.1 Deployment Strategy

2.5.2 Project Structure

3. Reference Implementation

3.1 Environment and Deployment

3.1.1 Workspace Setup

3.1.2 Silver Schema

3.1.3 Project Structure and CI/CD

3.2 Connectors

3.2.1 ServiceNow

3.2.2 GitHub

3.2.3 GitLab

3.2.4 SonarQube

3.2.5 Semgrep

3.2.6 Dependency-Track

3.2.7 TruffleHog

3.2.8 Vulnerability Enrichment

3.3 Analytics and Serving

3.3.1 Application-Repository Mapping

3.3.2 Application Risk Scoring

3.3.3 Remediation and Compliance Metrics

3.3.4 Vulnerability Trends

3.3.5 Risk Prediction Model

Conclusion

Thesis Outcomes and Contributions

Limitations

Future Work

References

- AXELOS. (2019). *ITIL foundation: ITIL 4 edition*. TSO (The Stationery Office). (Cit. on pp. 12, 13, 15).
- Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress. (Cit. on p. 16).
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press. (Cit. on p. 16).
- Gartner. (2024). *Market share: IT service management, worldwide, 2024*. Retrieved April 11, 2026, from <https://www.servicenow.com/blogs/2025/no-1-6-tech-workflow-segments> (cit. on p. 13).
- GitHub. (2024). *GitHub REST API documentation*. Retrieved April 11, 2026, from <https://docs.github.com/en/rest> (cit. on p. 16).
- GitHub. (2025). *Octoverse 2025: The state of open source*. Retrieved April 11, 2026, from <https://octoverse.github.com/> (cit. on p. 16).
- GitLab. (2024). *GitLab REST API documentation*. Retrieved April 11, 2026, from <https://docs.gitlab.com/ee/api/rest/> (cit. on p. 16).
- JetBrains. (2025). *The state of CI/CD in 2025*. Retrieved April 11, 2026, from <https://blog.jetbrains.com/teamcity/2025/10/the-state-of-cicd/> (cit. on p. 17).
- Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press. (Cit. on p. 16).
- Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill. (Cit. on p. 15).
- ServiceNow. (2024a). *CMDB instance API reference*. Retrieved March 7, 2026, from <https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/cmdb-instance-api.html> (cit. on pp. 13, 14).
- ServiceNow. (2024b). *Scripted REST APIs*. Retrieved April 11, 2026, from https://www.servicenow.com/docs/bundle/yokohama-api-reference/page/integrate/custom-web-services/concept/c_CustomWebServices.html (cit. on p. 14).
- ServiceNow. (2024c). *Table API reference*. Retrieved March 7, 2026, from https://www.servicenow.com/docs/bundle/xanadu-api-reference/page/integrate/inbound-rest/concept/c_TableAPI.html (cit. on p. 14).
- Skoulikari, A. (2024). *Learning git: A hands-on and visual guide to the basics of git*. O'Reilly Media. (Cit. on p. 16).
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson. (Cit. on p. 15).
- Stack Overflow. (2025). *2025 stack overflow developer survey*. Retrieved April 11, 2026, from <https://survey.stackoverflow.co/2025/> (cit. on pp. 16, 17).
- Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on p. 12).

- Williams, R., & Gonzalez, K. (2021). *3 steps to improve IT service view CMDB data quality*. Retrieved April 11, 2026, from <https://www.gartner.com/en/documents/3994127> (cit. on p. 15).
- Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media. (Cit. on p. 16).

Appendices

A. Generative AI Use Disclosure

A.1 Text Content

A.2 Diagrams and Figures

A.3 Source Code

A.4 Requirements Specification